

Innovation Feature

NETWORKING INTELLIGENCE TOOL

Mariane McGrath – G00473468

Applied Databases – Atu Galway-Mayo – May/ 2026

Table of Contents

1. Rationale	2
1.1 Query Logic	3
2. How It Works	4
2.1 Enter an Attendee ID	4
2.2 Example Scenario	4
3. Technical Implementation	6
3.1 Graph-Based Relationship Modelling (Neo4j)	6
3.2 Filtering Logic	6
3.3 Data Enrichment (Neo4j + MySQL Integration)	6
3.4 DAO Pattern and Code Structure	6
3.5 Scalability and Design Considerations	7
3.6 Format Note: Neo4j Database	7
4. Limitations	7
5. GDPR Considerations	8
6. Packages Required	8
7. Testing on the VM	9
8. Business Value	10
8.1 For attendees	10
8.2 For organisers	10
9. References	11
9.1 AI Assistance Declaration	12

Innovation Feature: Graph-Based Networking Recommendations for Conference Attendees

1. Rationale

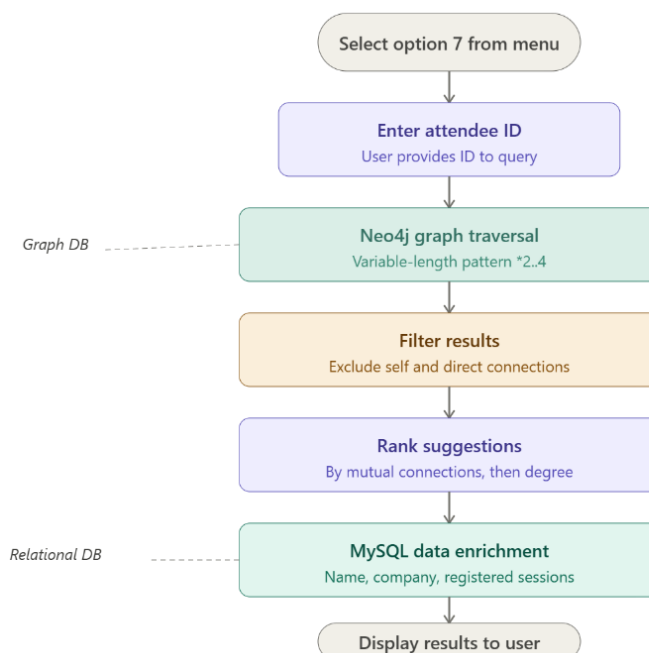
Considering the importance of professional networking in conferences, this project introduces a feature that supports the identification of meaningful connections between attendees.

Professional relationships often grow through shared interests and mutual contacts. Therefore, this project proposes a networking menu that recommends connections through **friend-of-a-friend (second-degree) relationships**, exploring between **two to four levels of separation**.

Using **Neo4j graph relationships**, this innovation demonstrates how graph databases can be used not only to store connections, but to actively generate meaningful networking opportunities for conference attendees. To support organic interactions, the feature also displays the **conference sessions each suggested connection is attending**, enabling shared interests and natural conversation starters to be identified before attendees meet.

This feature represents a **simple form of link prediction within a graph structure**.

Figure 1 — Innovation feature



1.1 Query Logic

The suggested connections feature is implemented using **Neo4j graph traversal**. Given a selected attendee, the system searches for other attendees within **two to four degrees of separation** using a variable-length relationship pattern (*2..4).

The query excludes:

- The selected attendee
- All direct connections

This ensures that only **new and relevant networking suggestions** are returned.

Where multiple paths exist between two attendees, the **minimum path length** is selected to represent the closest degree of separation.

To prioritise recommendations, the system calculates the **number of mutual connections** shared between the selected attendee and each suggested connection.

Mutual connections are identified using the pattern:

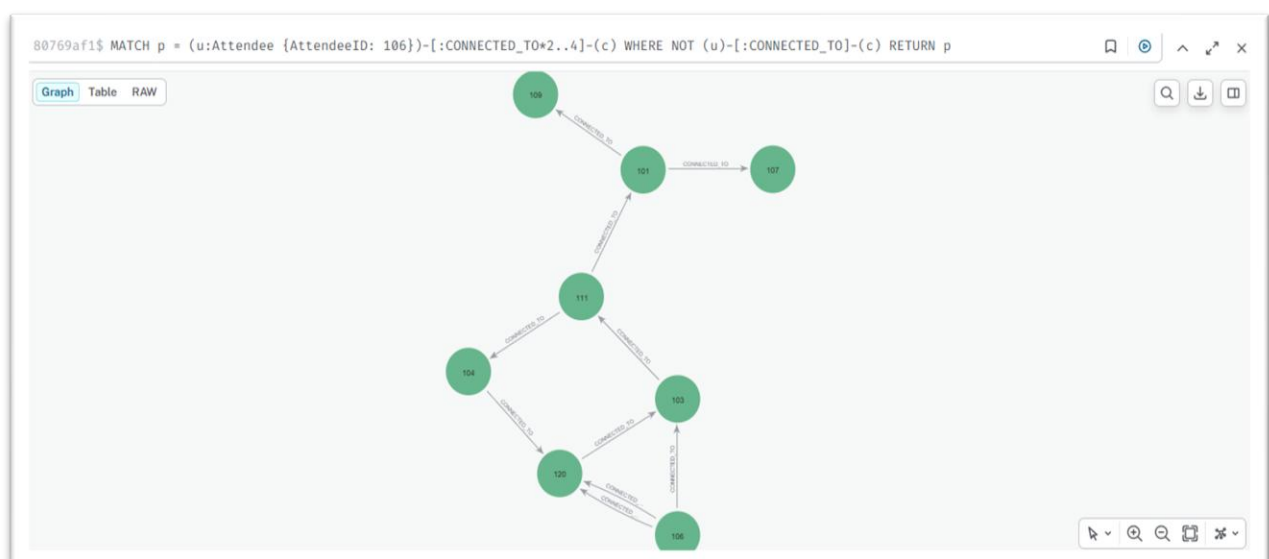
(a)-[:CONNECTED_TO]-(mutual)-[:CONNECTED_TO]-(suggested)

Results are ranked by:

1. Number of mutual connections (descending)
2. Degree of separation (ascending)

Finally, **MySQL** is used to enrich results with attendee details such as name, company, and registered sessions, demonstrating a **hybrid integration of relational and graph databases**.

Figure 2 — Neo4j graph traversal — degrees of separation



2. How It Works

When a user selects the **Networking Suggestions** option (Option 7) from the main menu, the system guides them through the following process:

2.1 Enter an Attendee ID

The user is prompted to enter an attendee ID. The system then performs a traversal of the **Neo4j graph database** to identify attendees within **2–4 degrees of separation**.

Direct connections and the selected attendee are excluded to ensure only **new networking opportunities** are presented.

The system displays:

- Suggested attendees
- Degree of separation
- Number of mutual connections
- Attendee details from MySQL
- Conference sessions each suggested attendee is registered for

2.2 Example Scenario

For example, when **Ava Murphy (ID 101)** requests suggestions, the system identifies **Noah Doyle (ID 103)** as a suitable connection.

Both attendees are registered for the **Graph Modelling Workshop**, creating an immediate and natural reason for engagement. This shared context transforms a cold introduction into a meaningful networking opportunity.

[SEE NEXT PAGE]

Figure 3 — Feature Output

- Suggested Connections

```
Conference Management
-----

MENU
=====
1 - View Speakers & Sessions
2 - View Attendees by Company
3 - Add New Attendee
4 - View Connected Attendees
5 - Add Attendee Connection
6 - View Rooms
7 - Networking Insights
x - Exit application
Choice: 7
Enter Attendee ID : 101

Suggested connections for Ava Murphy
(Based on shared connections in the network)
-----
Rank | Name | Mutual | Degree
-----
1 | Noah Doyle | 3 | 2
  Company: NeoRetail
  Sessions: Graph Modelling Workshop, Secure API Design

2 | Sean Dempsey | 3 | 3
  Company: GreenGrid
  Sessions: Energy Dashboards at Scale

3 | Emma Walsh | 2 | 2
  Company: GreenGrid
  Sessions: AI in Healthcare Operations, Energy Dashboards at Scale

4 | Jack Kelly | 2 | 3
  Company: FinPilot
  Sessions: Building Reliable ETL Jobs, FinTech Risk Signals

5 | Chloe Hunt | 1 | 2
  Company: EduSpark
  Sessions: EdTech Product Analytics

Tip: These attendees share mutual connections – consider facilitating an introduction.
```

- Key connectors

```
Key Connectors at this Conference
(Attendees ranked by number of connections)
-----
Rank | Name | Company | Connections
-----
1 | Ava Murphy | DataNova | 5
2 | Noah Doyle | NeoRetail | 3
3 | Grace Power | NeoRetail | 3
4 | Sean Dempsey | GreenGrid | 3
5 | Jack Kelly | FinPilot | 3
6 | Mia O'Brien | EduSpark | 2
7 | Liam Byrne | CloudSprint | 2
8 | Emma Walsh | GreenGrid | 2
9 | Cian Roche | CloudSprint | 1
10 | Sophia Ryan | MedAxis | 1

Tip: These are your most networked attendees – ideal for panels, table leads, or introductions.
```

3. Technical Implementation

3.1 Graph-Based Relationship Modelling (Neo4j)

Attendee relationships are modelled in Neo4j using `CONNECTED_TO` relationships. This graph structure allows efficient traversal of indirect relationships such as friend-of-a-friend connections, which would be significantly more complex to implement using purely relational joins.

A bounded traversal is performed with a variable-length relationship pattern, ensuring accurate calculation of degrees of separation while avoiding unnecessary traversal depth.

3.2 Filtering Logic

To maintain result relevance, the query includes filtering conditions to:

- Exclude the selected attendee.
- Exclude all direct connections.
- Return only indirect connections (degree ≥ 2)

This ensures that all results represent **new and meaningful networking opportunities**.

3.3 Data Enrichment (Neo4j + MySQL Integration)

Neo4j is used to identify relationships, while MySQL stores structured attendee data. Suggested attendee IDs are enriched by joining the attendee, company, registration, and session tables.

This hybrid design allows each database technology to focus on its strengths: relationship traversal in Neo4j and structured data management in MySQL.

3.4 DAO Pattern and Code Structure

The feature is implemented using the **Data Access Object (DAO) pattern** within a dedicated file (`dao.py`). This separates database logic from application logic.

Benefits include:

- Improved maintainability
- Easier testing

- Reusable database queries

The `suggest_connections()` method performs traversal, filtering, ranking, and data retrieval in a structured and reusable manner.

3.5 Scalability and Design Considerations

Using a graph database enables the system to scale effectively as the number of attendees increases. Limiting traversal depth to **2–4 degrees** ensures:

- Controlled query performance
 - Relevant recommendations
 - Avoidance of distant or weakly related connections
-

3.6 Format Note: Neo4j Database

During development, JSON imports caused compatibility issues with the Neo4j Browser version used. To ensure reliable imports, **Cypher was used** instead, as recommended by Copilot.

Testing confirmed that Cypher consistently imported all nodes and `CONNECTED_TO` relationships correctly across both Neo4j Aura and local Desktop environments.

To import the data, open the Neo4j Browser, drag the Cypher file into the query pane, and run it.

4. Limitations

The system has several intentional limitations:

- All connections are treated equally (unweighted)
 - Session attendance is used as a proxy for shared interest
 - No industry-based filtering is applied
 - Attendees with few connections may receive limited suggestions
-

5. GDPR Considerations

This feature processes personal data under **GDPR** as enacted in Ireland through the **Data Protection Act 2018**.

In a production environment, a lawful basis such as **legitimate interest (Article 6(1)(f))** would be appropriate, subject to a balancing test. Data visibility is intentionally limited to session titles, with no sensitive personal data exposed.

Attendees would be informed through a transparent privacy notice at registration, ensuring compliance with Articles 13 and 14 GDPR.

6. Packages Required

- neo4j
- mysql-connector-python
- python-dotenv

Install Command:

```
pip install neo4j==5.26.0 mysql-connector-python==9.1.0 python-dotenv==1.0.1
```

7. Testing on the VM

Comprehensive test cases validate error handling, traversal accuracy, and data enrichment across Neo4j and MySQL.

Figure 4 — Neo4j and MySQL Verification

```
conference management
-----

menu
====
1 - view speakers & sessions
2 - view attendees by company
3 - add new attendee
4 - view connected attendees
5 - add attendee connection
6 - view rooms
7 - networking insights
x - exit application
choice: goodbye!

--- END OUTPUT ---

--- STDERR ---

--- END STDERR ---

✅ Found: suggested connections

✅ TEST PASSED

=====
TEST SUMMARY
=====
✅ Passed: 12
❌ Failed: 0
📊 Total: 12
=====
```

8. Business Value

8.1 For attendees

The feature:

- Reduces networking friction.
- Supports organisers in facilitating introductions based on shared session interests.
- Reveals hidden networking opportunities.

8.2 For organisers

The feature:

- Identifies key connectors.
- Enables data-driven introductions.
- Enhances conference ROI through meaningful engagement.
- Provides visibility into attendee engagement and network structure.

9. References

Section 1. Innovation Feature Rationale

[Neo4j Graph Database Concepts — Nodes, Relationships and Properties](#)

[Link Prediction — Graph Proximity and Social Networks](#)

Section 1.1 Query Logic

[Neo4j Cypher Manual — Variable-Length Patterns](#)

[Cypher Variable-Length Relationships by Example](#)

Section 3.1 Graph Based Relationship Modelling (Neo4j)

[Neo4j — Comparing Relational to Graph Databases](#)

[Neo4j Cypher Manual — Variable-Length Patterns and Bounded Traversal](#)

Section 3.2 Filtering Logic

[Neo4j APOC — Expand Paths with Config, minLevel and maxLevel](#)

Section 3.3 Data Enrichment (Neo4j + MySQL Integration)

[Neo4j — Graph Database vs Relational Database](#)

[Performance of Graph and Relational Databases in Complex Queries](#)

Section 3.4 DAO Pattern and Code Structure

Data Access Object Design Pattern

[DAO Design Pattern and Separation of Logic](#)

[DAO and DTO Patterns in Python](#)

Section 3.5 Scalability and Design Considerations

[Neo4j Cypher Manual — Controlling Traversal Depth and Performance](#)

[Neo4j APOC — Bounded Path Expansion and Scalability](#)

Section 5. GDPR Considerations

[Article 6 GDPR — Lawfulness of Processing](#)

[Data Protection Commission Ireland — Guidance on Legal Bases for Processing Personal Data](#)

[Data Protection Commission Ireland — Right to Be Informed, Articles 13 and 14 GDPR](#)

[Article 13 GDPR — Information to Be Provided Where Personal Data Are Collected](#)

Section 8. Business Value

8.1 For Attendees

[Neo4j — Graph Databases for Relationship Traversal and Data-Driven Insights](#)

[Network Analysis — Identifying Key Connectors and Bridges in a Graph](#)

[Measuring Conference ROI — Relationship Building and Data-Driven Engagement Outcomes](#)

8.2 For Organisers

[The Science of Networking — Reducing Social Friction and Shared Interest Connections at Conferences](#)

[Conference Networking — Helping Attendees Make Meaningful Connections Through Shared Interests](#)

[Measuring Conference ROI — Attendee Engagement and the Value of Networking](#)

9.1 AI Assistance Declaration

During development, AI tools (ChatGPT, DeepSeek and Claude AI) were used for the following purposes:

- Debugging MySQL import issues related to PowerShell vs. Command Prompt encoding conflicts
- Generating test cases for edge conditions
- Refactoring Neo4j connection error handling

All AI-generated code was reviewed, tested, and integrated by the author. No core logic was generated entirely by AI.

Declared: May 2026 — Mariane McGrath